SESSION
MANAGEMENT

renam singla
Posted on Oct 3

❤️ 2 🦄 1 🤯 1 🙌 1 🔥 1

Session Management- Cookies, Session and JWT

#express #cookie #webdev #jwt

Since every request to a server is a new request, so for the server to identify whether the request came from the same browser/ user, we use SESSION MANAGEMENT.

In order to identify the user, we use the concept of ***Cookies, Session, and JWT***

COOKIES

The cookies , is a way to identify whether the user is a new user or has visited the website before.

- When a client sends a "sign in" request to the server using a form lets say, in response to that server generates the personalised or generic response .
- With the response, the server sends a "cookie", an additional data to the client which stores information about the user which helps to distinguish the user.
- Next time , whenever the user makes a `get` request to the server for that `PORT` it will also give its `cookie` with the `request` to the server , which says, *i am the same user that signed in before , hence helps for identification of the authenticated client.*
- Basically cookie is a medium through which a user can remain logged in as server has information of the user through cookie.

How to set a Cookie?

- Cookie stores the user information key-value format, *the cookie can be set by-*

```
//user, id and info is an object of cookie
res.cookie('user','123',{
            httpOnly: false, //it allows access to the javascript
            secure: false, //allow over HTTP
            sameSite: 'Strict',
            maxAfe: 65480000 //in sec, it gives the expire time of the c
            });
```

- The another way to set a cookie is-
  - This is an object of user-data containing the information of the user

```
const userdata= {
            username,
            id:123
            };
```

- This `userdata` object then further is `JSON.stringify` to store is a cookie with the help of function `res.cookie()`

```
res.cookie('user', JSON.stringify(userdata),{
            httpOnly: false, //it allows access to the javascript
            });
```

Here, the object of javascript needs to be changed as a string in order to store it in cookie, as it is not a primitive data type.

SO, LETS SEE HOW WILL COOKIES WORK IN CONTEXT OF AN APPLICATION!!

1. Start a Server -

To start a server , we need to install `npm` package and express and further use `app.listen()` function.

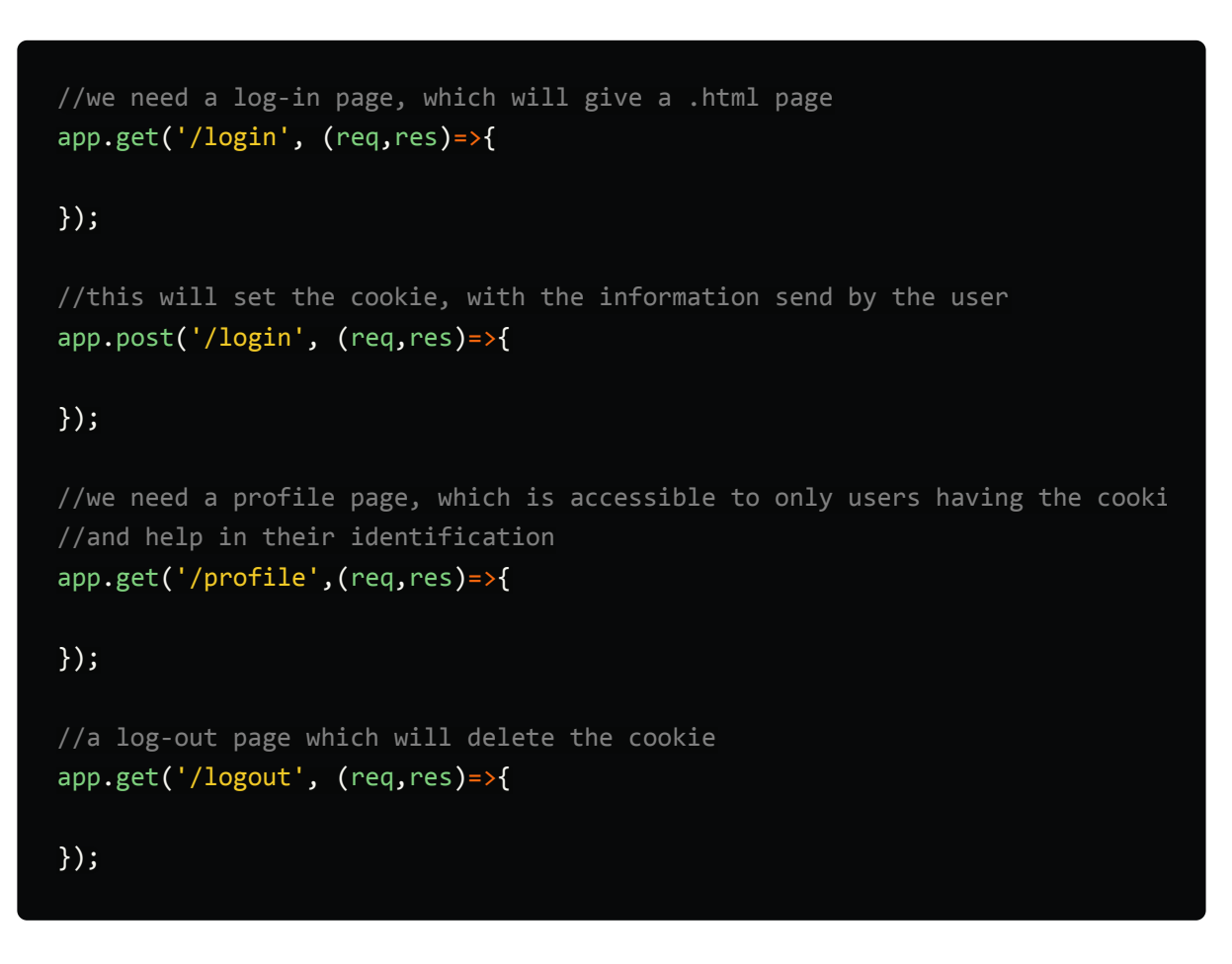
For in detail understanding refer to the previous blog- https://dev.to/renam_singla_fb52a400f07e/expressjs-for-beginners-get-post-serving-static-files-577d

2. Declaring the Functionality of the Application

It is important to know what the application will do and understanding of its flow before building it. So let's start with declaring our functionality, request and the working of the app.

```
//we need a log-in page, which will give a .html page
app.get('/login', (req,res)=>{

});

//this will set the cookie, with the information send by the user
app.post('/login', (req,res)=>{

});

//we need a profile page, which is accessible to only users having the cooki
//and help in their identification
app.get('/profile',(req,res)=>{

});

//a log-out page which will delete the cookie
app.get('/logout', (req,res)=>{

});
```

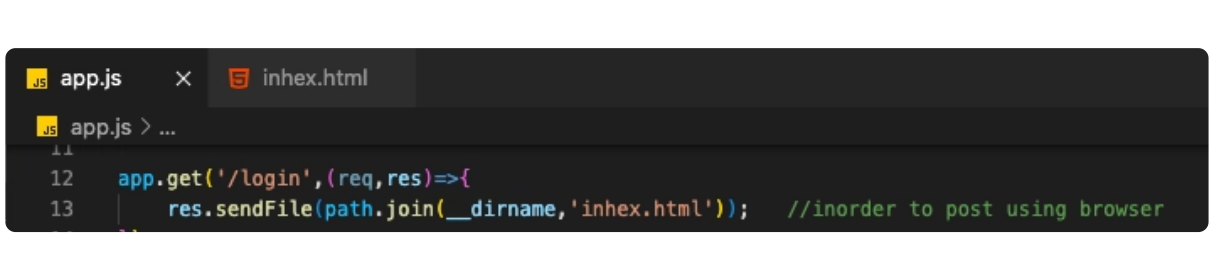
FLOW OF APPLICATION-



3. Create a "LOGIN" page -

Through this page, client can `sign in` to the application using `post` request and with it a cookie will be created and send to the browser by the server.

To access this page on browser, we create a `GET` REQUEST on the server , which will send this `file` in response using `res.sendFile` and `path.join()` function



```
app.get('/login',(req,res)=>{
    res.sendFile(path.join(__dirname,'inhex.html'));   //inorder to post using browser
```

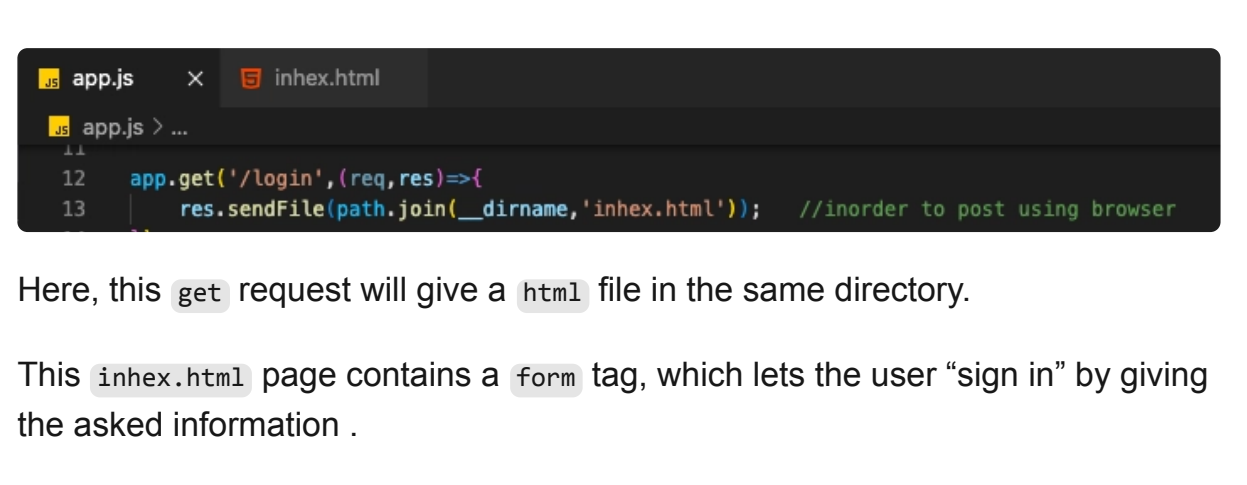
Here, this `get` request will give a `html` file in the same directory.

This `inhex.html` page contains a `form` tag, which lets the user "sign in" by giving the asked information .

```
<body>
    <h1>COOKIES</h1>
    <form action="/login" method="POST">
        <input type="text" name="username" placeholder="add username">
        <button type="submit">login</button>
    </form>
</body>
```

Further, submitting the form, will send a `post` request through browser, through which later a cookie will be set.

So , our `login` page will look like-



Request URL http://localhost:4441/login
Request Method GET
Status Code 304 Not Modified
Remote Address [::1]:4441
Referrer Policy strict-origin-when-cross-origin

But , if cookie already exists, then user must be redirected to the profile-

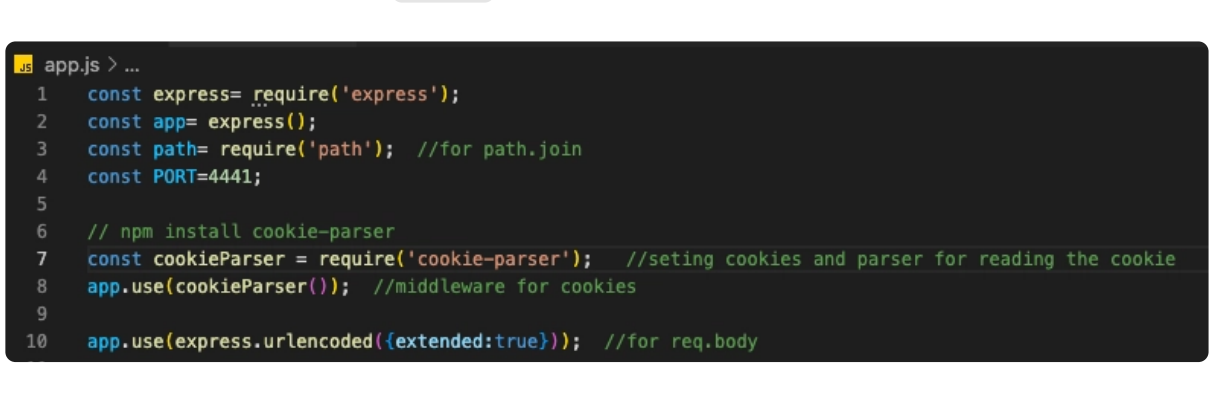
```
if (req.cookies.user) {
        return res.redirect('/profile');
    }
```

4. Set a Cookie-

To set a cookie, we first need to install the cookie package from `npm` , here we are going to use `cookie-parser` package.

```
npm install cookie-parser
```

And then, require it in the `app.js` file

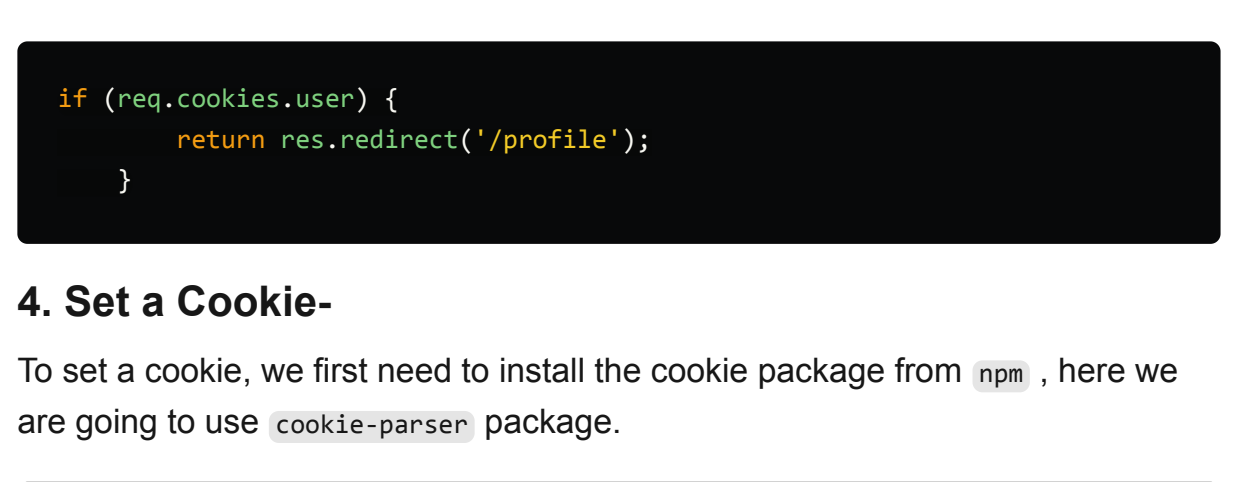


```
const express= require('express');
const app= express();
const path= require('path');  //for path.join
const PORT=4441;

// npm install cookie-parser
const cookieParser = require('cookie-parser');   //seting cookies and parser for reading the cookie
app.use(cookieParser());  //middleware for cookies

app.use(express.urlencoded({extended:true}));  //for req.body
```

As discussed initially , to set a cookie, we use `res.cookie()` function.

- Now, we will first require the "username" send by the user by the form.
- Then, create an object for cookie data using that username.
- Which will further help to set a cookie after changing it to a string

…when the `post` request was made on the submission of the `form`



```
app.post('/login',(req,res)=>{
    const{username}= req.body;

    // creating object for cookie
    let userData={
        username,
        count:0
    }

    res.cookie('user',JSON.stringify(userData),{
        httpOnly:true
    });     //setting a cookie with an object of key 'user' and value stringified of 'userData'
// all the above done using form in index.html

    res.redirect('/profile');   //after submit it redirects to profile page as response
})
```

And since the cookie is set now, the user can have access to `/profile` URL path

5. Access to "/profile" and Working of it-

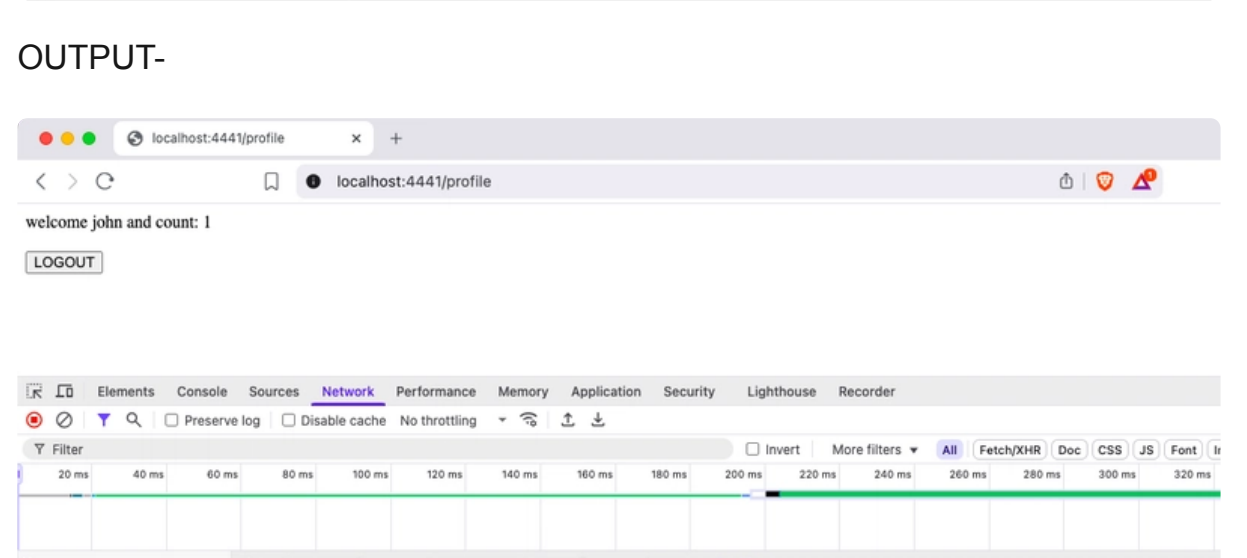
Firstly for any user to access `'profile` path , it is mandatory to have a cookie, which means they have logged in , otherwise the user should be redirected to the `/login` path

- `req.cookies` : command helps to read the cookie, by the name- user

```
app.get('/profile', (req,res)=>{

                if(!req.cookies.user){
                    res.redirect('/login');
                }
});
```

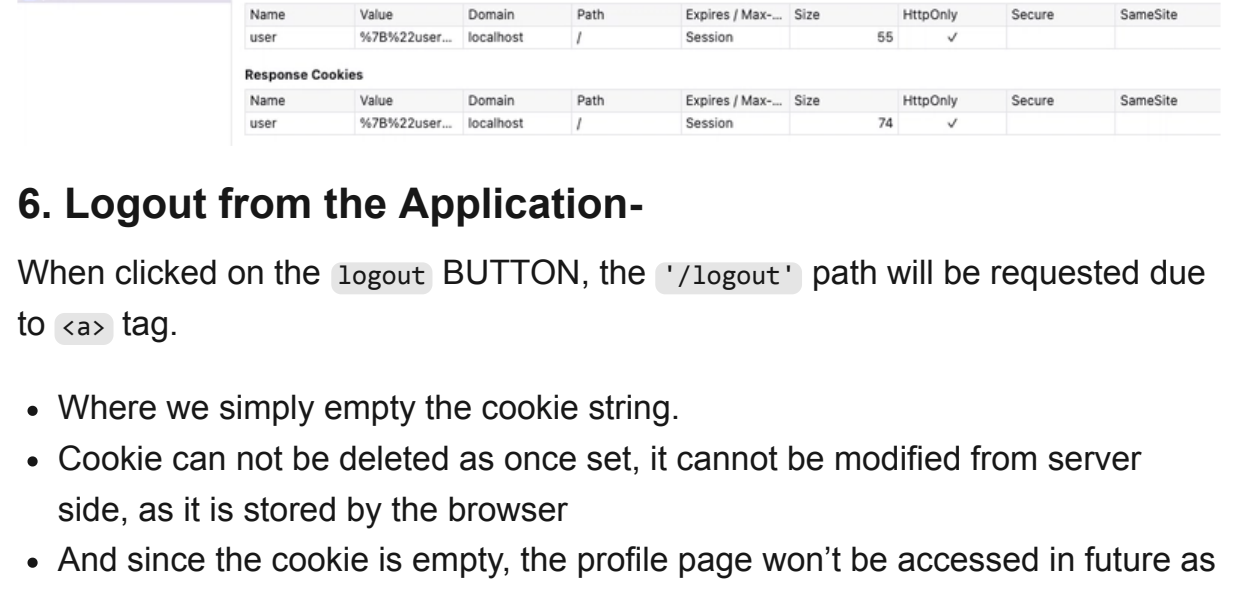
If cookie exists-

- We can have the username on the profile with number of times the user visited/ refreshed the profile url path
- For `js` to require the username from the cookie, we need to `parse` the cookie
- With the above response in `res.send()` , we will also send a `logout` button (having `/logout` path) if user wants to log out from the application



```
app.get('/profile',(req,res)=>{
    console.log(req.cookies);

//    making sure cookie exists
    if(!req.cookies.user){
        return res.redirect('/login');
    }

    let userData= JSON.parse(req.cookies.user);  //parsing the cookie to get username and count

    userData.count++;

    // after increasing the count, setting up the same cookie again
    res.cookie('user',JSON.stringify(userData),{
        httpOnly:true
    });
    console.log(userData.count);

    res.send(`welcome ${userData.username} and count: ${userData.count}
        <br>
        <br>
        <a href='/logout'>
            <button>LOGOUT</button>
        </a>
    `);  //r                lient side
})
```

OUTPUT-



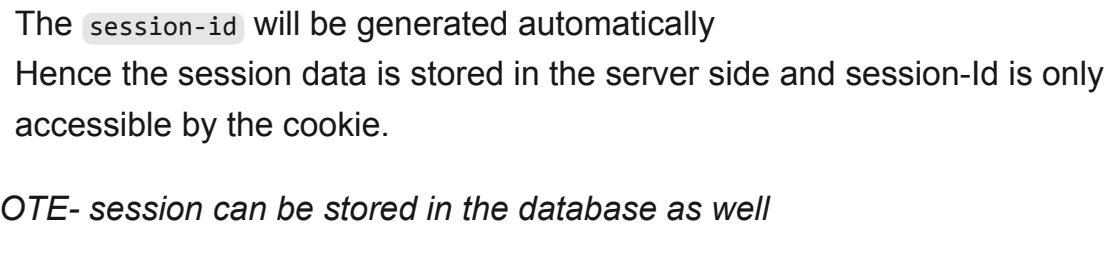
welcome john and count: 1

LOGOUT

6. Logout from the Application-

When clicked on the `logout` BUTTON, the `'/logout'` path will be requested due to `<a>` tag.

- Where we simply empty the cookie string.
- Cookie can not be deleted as once set, it cannot be modified from server side, as it is stored by the browser
- And since the cookie is empty, the profile page won't be accessed in future as it requires `username` and further redirect to the `/login` .



```
app.get('/logout',(req,res)=>{
    res.cookie('user',"");     //setting up an empty cookie
    res.redirect('/login');  //after logout, redirecting to login page
})
```

With the cookies, there is an issue that we have not addressed yet, that they can be manipulated y anyone and hence the user can be changed without any restriction.

To stop this, we use **SESSION.**

SESSION

- The session has a "session map" which will store the object of the session-data, and it will send an "id" in respond which will be stored in cookie rather than the data.
- Therefore, no one will have access to the authentication data and it cannot be manipulated anymore.
- The `session-id` will be generated automatically
- Hence the session data is stored in the server side and session-Id is only accessible by the cookie.

NOTE- session can be stored in the database as well

1. How to use express-session?

Firstly install `express-session` npm package, in the terminal-

```
npm i express-session
```

Then require it in the `js` file-

```
const session= require('express-session');
app.use(session({
    secret: 'notknownbytheuser', //for id generation, secret is required
    resave: false, //do not change the data
    saveUninitialized: true
    //irrespective of requirement everyone will get a session ID
}))
```

- Secret should not be shared with anyone
- `saveUninitialized` will create an id even before the user has login as default

The `express-session` do not need to use parser, it manages it by itself

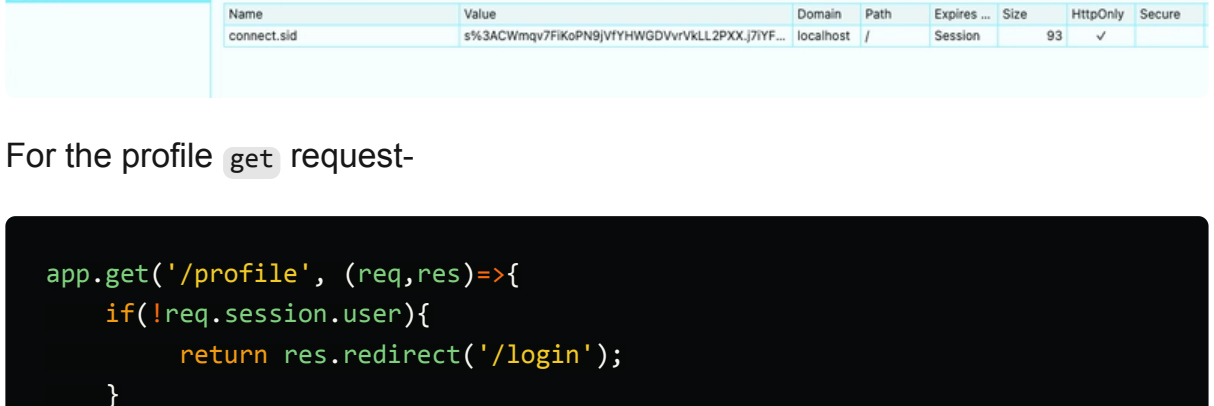
2. Set a session-

On our `login` page, to set a session-

- The user provides a username and the object of that is then required by the session using the `POST` request.
- To set the session and to read as well we use- `req.session.user`
- And in return it creates an unique id by itself

```
app.post('/login',(req,res)=>{
    const {username}= req.body;

    let userData={
        username,
        count:0
    }

    // setting the session- putting userData in the session
    req.session.user= userData;
    res.redirect('/profile');
})
```

The Output in the browser, will only show the "id"



welcome john with the count: 1

logout

For the profile `get` request-

```
app.get('/profile', (req,res)=>{
    if(!req.session.user){
        return res.redirect('/login');
    }

    req.session.user.count++;
    let userData= req.session.user;

    res.send(`welcome ${userData.username} with the count: ${userData.count}
        <br> <br>
        <a href='/logout'>
            <button>logout</button>
        </a>`)
})
```

3. Delete a session-

When clicked on the "logout" button, `/logout` path is requested-

```
app.get('/logout',(req,res)=>{
    req.session.destroy(function(error){
        res.redirect('/login');
    })
})
```

To delete, we use `req.session.destroy(function(error))` function which is an error first callback function.

If there is no error it will simply does perform the function otherwise will give an error

LASTLY,

if the server is refreshed, session map will loose its data, hence it is better to store sessions in a database.

JWT- JSON WEB TOKEN

JWT comprises of three things-

- Header: type and algorithm
- Payload: user information
- Signature: having header, payload and secret

We also need JWT secret.

JWT does not go with each request by itself like cookie, it needs to be send with the request.

Here, we will also be requiring cookie to store "token", hence install that by referring to the above blog.

1. Install JWT-

```
const jwt= require('jsonwebtoken');
```

JWT generates a "token" when provided the payload and the JWT secret

Therefore, we have to create a JWT secret as well-

```
const JWT_SECRET= "decneicnjvbjrgvd";
```

2. Generate a JWT -

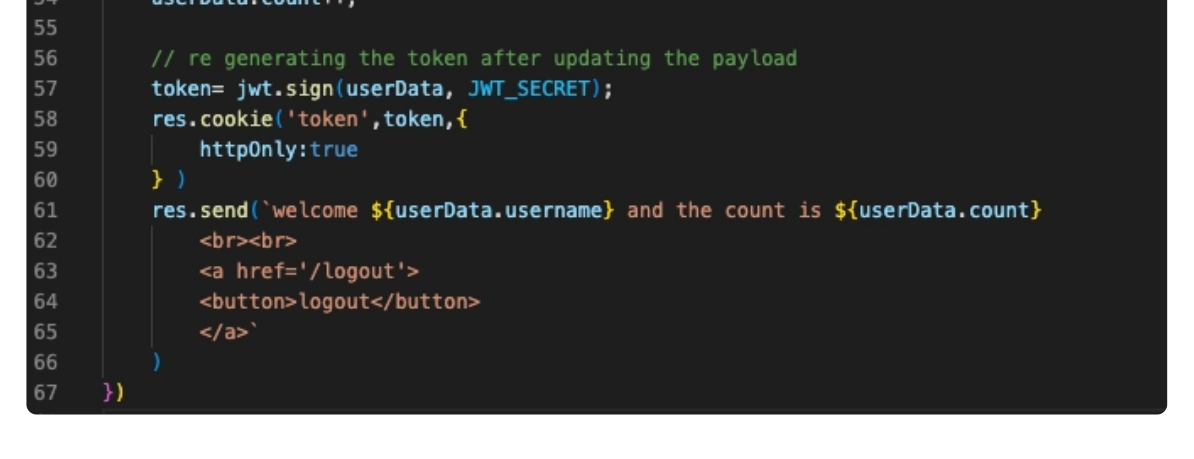
In the `post` request of the form when submitted the form, we first create a token by-

```
var token= jwt.sign(userData, JWT_SECRET);
```

And then this token is stored in the cookie using `CookieParser` as we did before

```
res.cookie('token',token,{
        httpOnly:true
    } )
```

So the `POST` request for `/login` looks like-



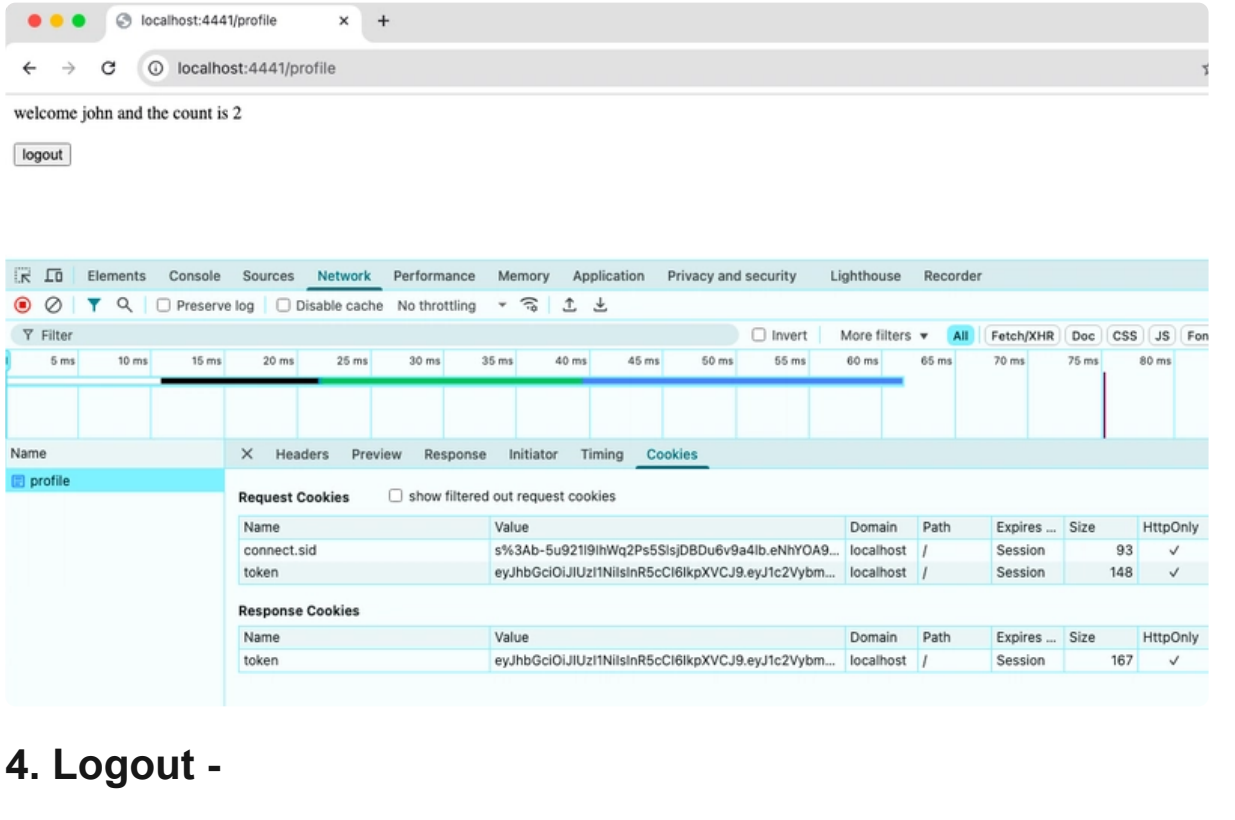
```
app.post('/login',(req,res)=>{
    const {username}= req.body;

    let userData= {
        username,
        count:0
    }

    var token= jwt.sign(userData, JWT_SECRET);
    res.cookie('token',token,{ //not using JSON.stringify because token is already a string
        httpOnly:true
    } )

    res.redirect('/profile');
})
```

3. Verification of the token -

Before giving access to the profile, it is important to verify that the token exists and if exists , does it matches the user.

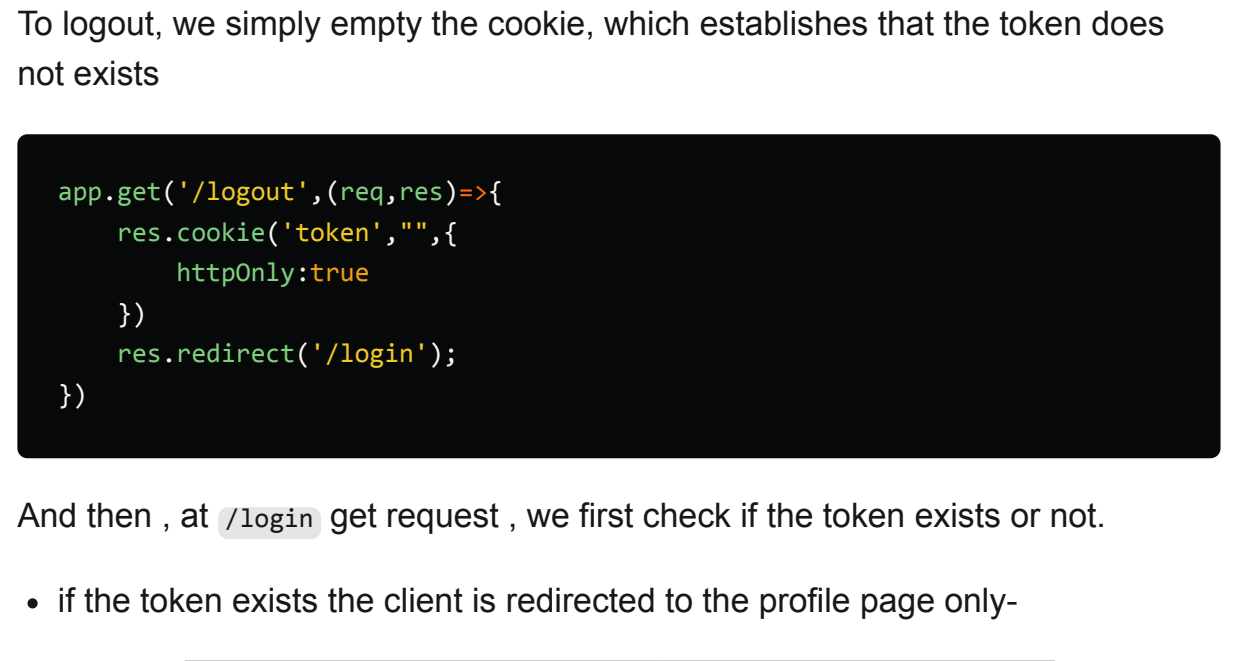
- first we require the token from the cookie, if it exists then we proceed
- The `userData` then is verified using the "token" and the "secret key"
- If it does not matches , the request will be redirected to the `/login`
- This is done using either error first callback function or using try-catch syntax

Now, after the token is verified, we proceed to show the profile to the client.



```
app.get('/profile',(req,res)=>{
    var token= req.cookies.token //not using JSON.parse because token is not an object
    if(!token){
        return res.redirect('/login');
    }

    var userData;
    try{
        userData= jwt.verify(token, JWT_SECRET);
    } catch(error){
        res.redirect('/login');
    }

    userData.count++;

    // re generating the token after updating the payload
    token= jwt.sign(userData, JWT_SECRET);
    res.cookie('token',token,{
        httpOnly:true
    } )
    res.send(`welcome ${userData.username} and the count is ${userData.count}
        <br><br>
        <a href='/logout'>
        <button>logout</button>
        </a>`
    )
})
```

OUTPUT ON THE BROWSER AND THE TOKEN IN THE COOKIE-



welcome john and the count is 2

logout

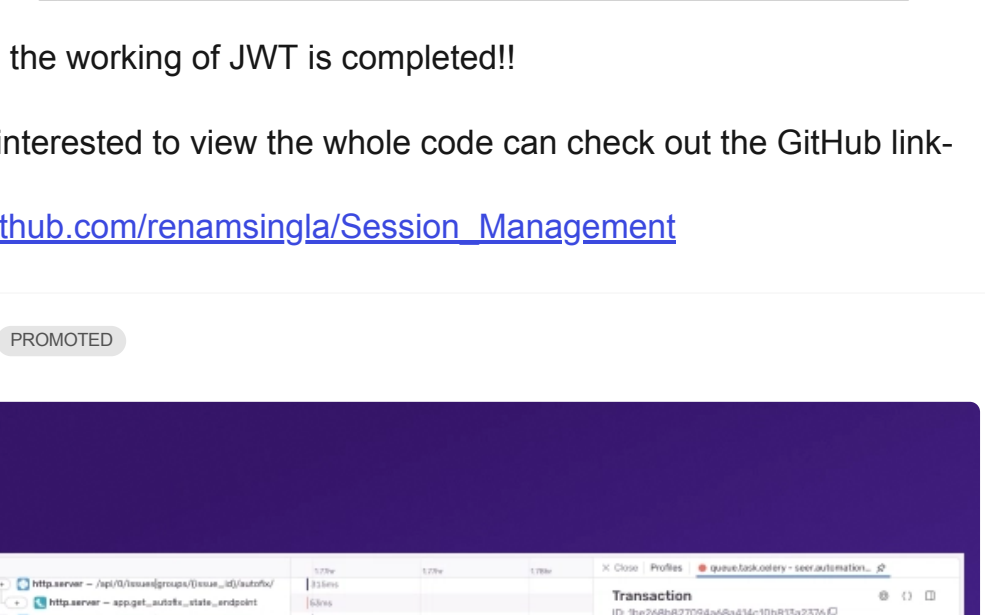
4. Logout -

To logout, we simply empty the cookie, which establishes that the token does not exists

```
app.get('/logout',(req,res)=>{
    res.cookie('token',"",{
        httpOnly:true
    })
    res.redirect('/login');
})
```

And then , at `/login` get request , we first check if the token exists or not.

- if the token exists the client is redirected to the profile page only-



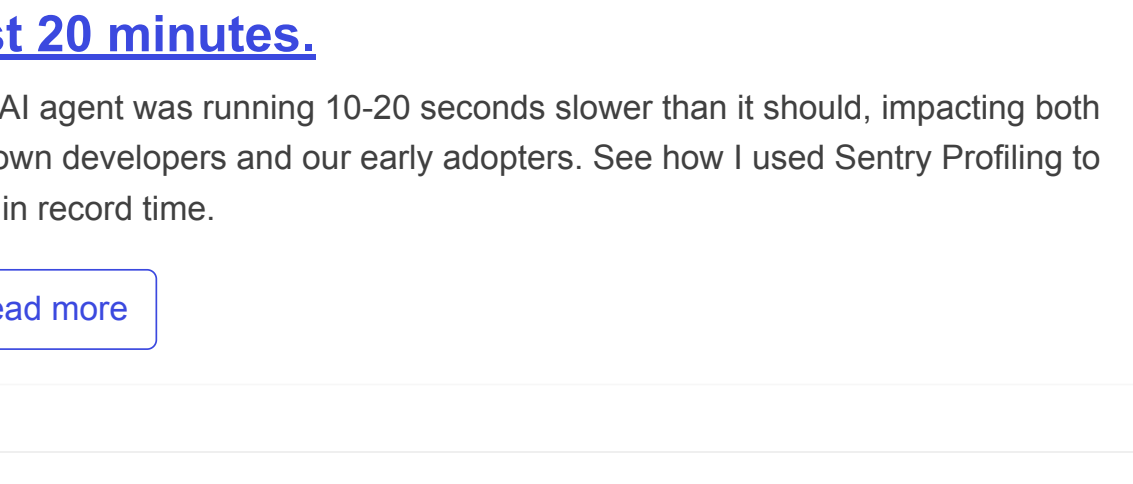
```
const express= require('express');
const app= express();
const path= require('path');
const PORT= 4441;

const cookieParser= require('cookie-parser')
app.use(cookieParser());

const jwt= require('jsonwebtoken');
const JWT_SECRET= "decneicnjvbjrgvd";

app.use(express.urlencoded({extended:true}));

app.get('/login', (req,res)=>{
    let token= req.cookies.token
    let userData;
    try{
        userData= jwt.verify(token, JWT_SECRET);
    } catch(error){
        return res.sendFile(path.join(__dirname,'index.html'));
    }
    res.redirect('/profile');
})
```

With this the working of JWT is completed!!

Anyone interested to view the whole code can check out the GitHub link-

https://github.com/renamsingla/Session_Management

Sentry PROMOTED



How I fixed 20 seconds of lag for every user in just 20 minutes.

Our AI agent was running 10-20 seconds slower than it should, impacting both our own developers and our early adopters. See how I used Sentry Profiling to fix it in record time.

Read more

Top comments (2)

Subscribe

Add to the discussion

Naman Arora • Oct 3

Singla ji kya aap mujhe tuition dedoge aapke notes toh badhe ache hai❤

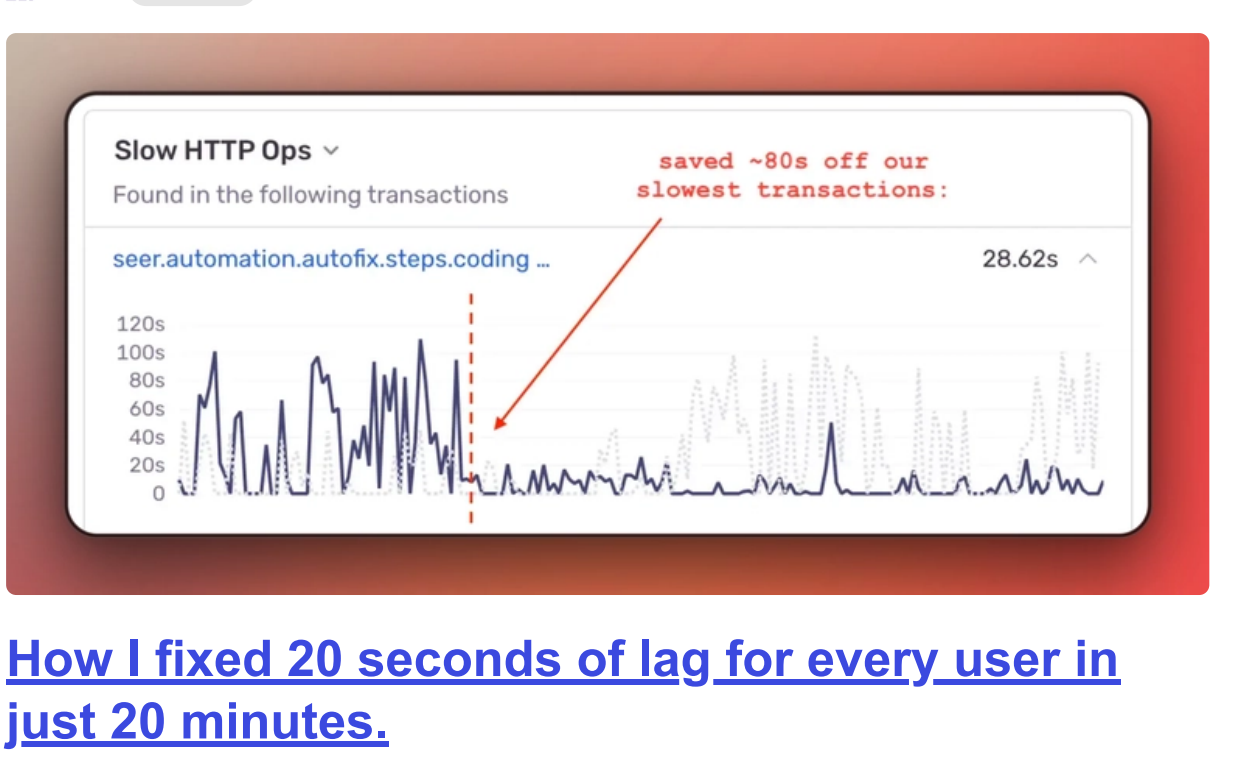
1 like Reply

Naman Arora • Oct 3

ohho the notes are super awesome😍😍😍😍

1 like Reply

Sentry PROMOTED



How I fixed 20 seconds of lag for every user in just 20 minutes.

Our AI agent was running 10-20 seconds slower than it should, impacting both our own developers and our early adopters. See how I used Sentry Profiling to fix it in record time.

Read more

renam singla
Follow
Aspiring software development engineer
JOINED
Sep 18, 2025

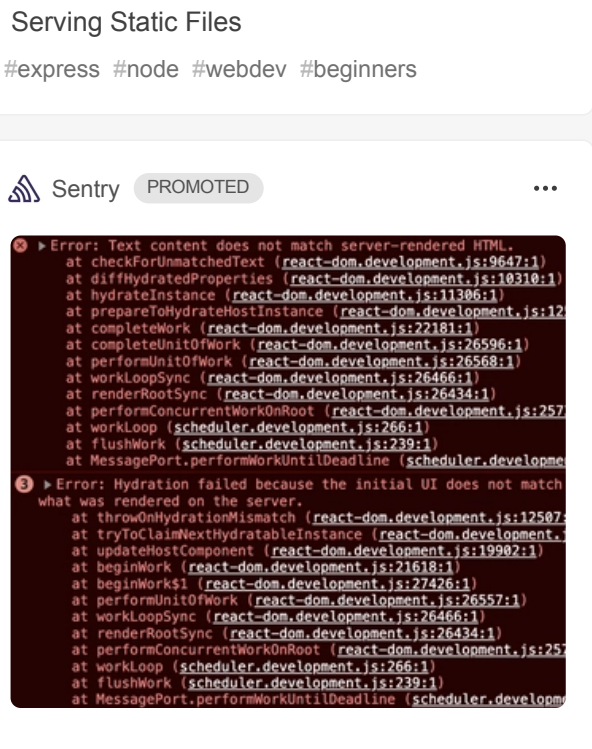
More from renam singla

WEBSOCKETS
#express #webdev #socket

Full-Stack TodoApp: Node.js, Express, MongoDB, and HBS Guide
#express #mongodb #webdev #fullstack

Express.js for Beginners: GET, POST & Serving Static Files
#express #node #webdev #beginners

Sentry PROMOTED



If seeing this in React makes you , get Sentry.

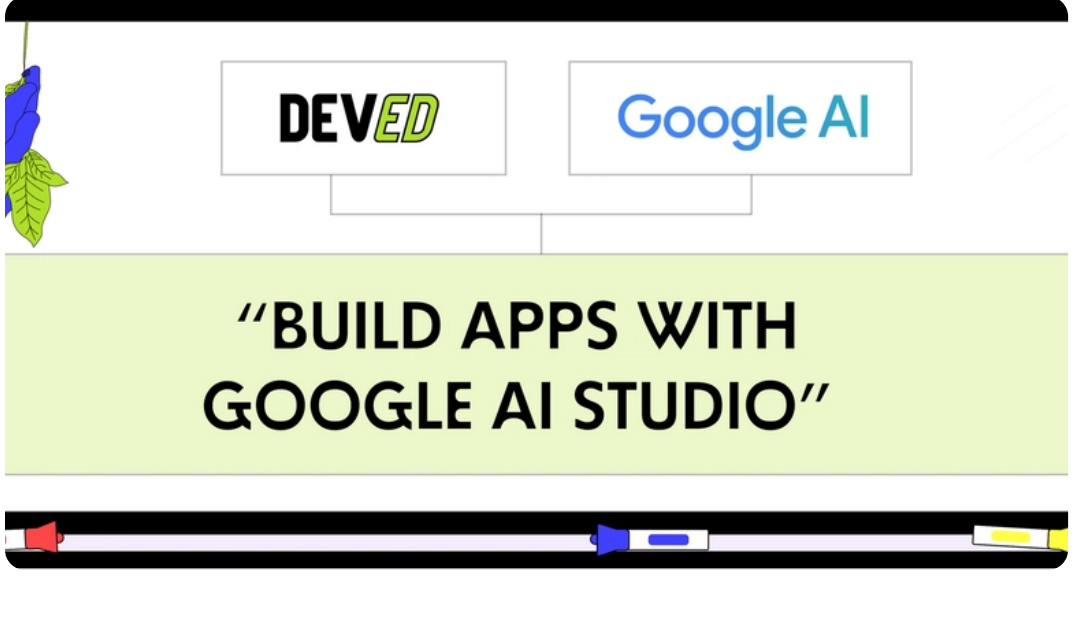
Try Sentry

DEV Education Tracks

Announcing the First DEV Education Track: "Build Apps with Google AI Studio"

The moment is here! We recently announced DEV Education Tracks, our new initiative to bring you structured learning paths directly from industry experts.

Dive in and Learn


DEVED
Google AI
"BUILD APPS WITH GOOGLE AI STUDIO"

DEV is bringing Education Tracks to the community. Dismiss if you're not interested ❤

Built on Forem — the open source software that powers DEV and other inclusive communities.
Made with love and Ruby on Rails. DEV Community © 2016 - 2025.